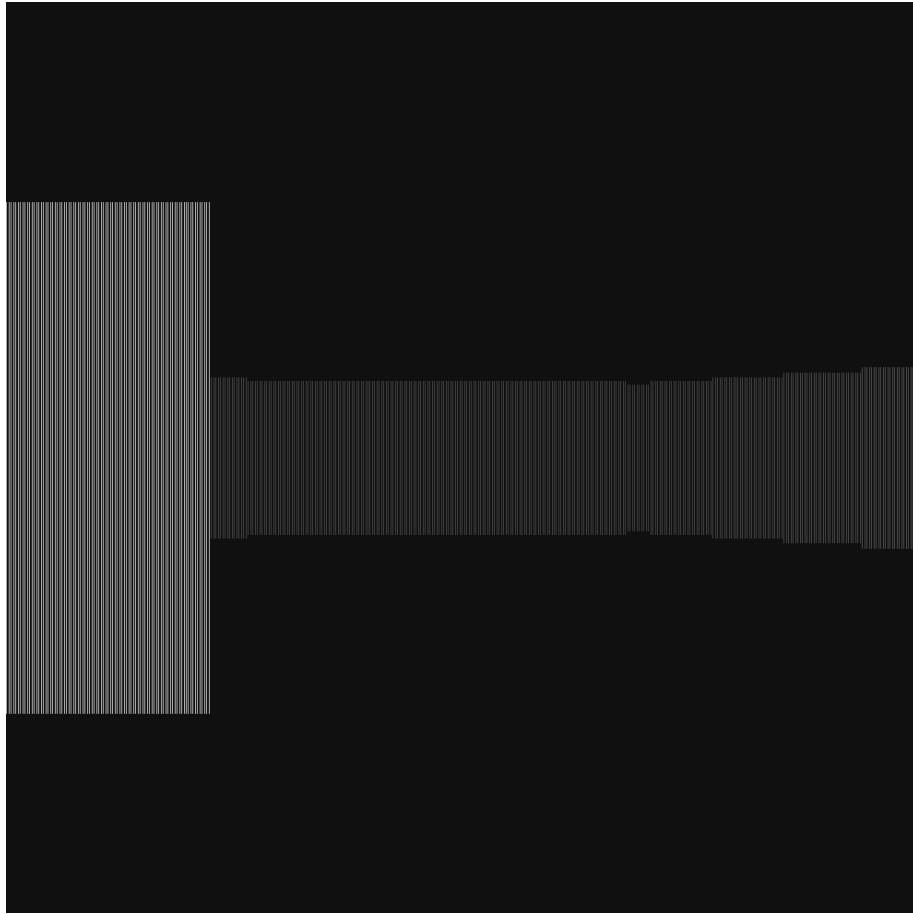


On Implementing Raycasting



ShadowNetter

Logic

- Camera Variables

- x position
- y position
- angle (view direction)
- FOV (field of view)

- Ray Variables

- angle per ray
- step size (distance increment per step)

Logic

- Raycasting

- ray count = $\text{FOV} / \text{angle_per_ray} + 1$
(the +1 ensures at least one ray)

- for each ray:

compute ray angle:

$\text{ray_angle} = \text{player.angle} - \text{FOV}/2 + \text{ray_index} * (\text{FOV} / (\text{ray_count} - 1))$

compute horizontal screen position:

$\text{screen_x} = \text{ray_index} * \text{screen_width} / \text{ray_count}$

convert ray angle to radians:

$\theta = \text{ray_angle} * \pi / 180$

step forward until collision or out of bounds:

$\text{ray_x} = \text{camera.x} + \cos(\theta) * \text{steps}$

$\text{ray_y} = \text{camera.y} + \sin(\theta) * \text{steps}$

stop if out of map or hitting a wall

compute line details:

base colour = 255

prevent division by zero: if $\text{steps} \leq 0 \rightarrow \text{steps} = \text{step_size}$

$\text{shade} = \text{base colour} / \text{steps}$

(lower shade aka darker, means object is further away)

$\text{line height} = \text{screen_height} / \text{steps} + 1$

Implementing

- Define structs
 - Camera
 - Line
- Define environmental variables
 - Screen height
 - Screen width
 - Step size
 - Angle per ray
- Main function
 - Initialize Camera
 - Initialize Map
 - Loop
 - Clear screen
 - (Optional) Camera Movement
 - Raycast from map
 - Display lines from Raycast function

Example Code

```
#include <math.h>
#include <stdbool.h>

// Define structs
typedef struct {
    float x;
    float y;
    float angle;
    int fov;
} Camera;

typedef struct {
    int x;
    int height;
    int colour;
} Line;

// Define environmental variables
const int SCREEN_HEIGHT = 10;
const int SCREEN_WIDTH = 20;
const float STEP_SIZE = 1;
const float ANGLE_PER_RAY = 0.5;

void Raycast(
    Camera camera,
    int map[SCREEN_HEIGHT][SCREEN_WIDTH],
    int ray_count,
    Line lines[ray_count]
);
```

Example Code

```
// Main function
int main(void) {
    // Initialize Camera
    Camera camera;
    camera.x = 0;
    camera.y = 0;
    camera.angle = 0;
    camera.fov = 3°;

    // Initialize Map
    int map[SCREEN_HEIGHT][SCREEN_WIDTH];
    for (int y = 0; y < SCREEN_HEIGHT; y++) {
        for (int x = 0; x < SCREEN_WIDTH; x++) {
            map[y][x] = 0;
        }
    }

    map[3][0] = 1;
    map[3][1] = 1;
    map[3][2] = 1;

    int ray_count = camera.fov / ANGLE_PER_RAY + 1;
    Line lines[ray_count];

    // Loop
    bool running = true;
    while (running) {
        // Clear screen
        // (Optional) Camera movement
        Raycast(camera, map, ray_count, lines);
        // Render()
    }

    return 0;
}
```

Example Code

```
void Raycast(
Camera camera, int map[SCREEN_HEIGHT][SCREEN_WIDTH],
int ray_count,
Line lines[ray_count]
) {
    for (int ray = 0; ray < ray_count; ray++) {
        float ray_angle = (
            player.angle - FOV/2.0 + ray_index * (FOV / (ray_count - 1.0))
        );
        float screen_x = ray * SCREEN_WIDTH / ray_count;
        float theta = ray_angle * M_PI / 180;

        int steps = 0;
        for (steps = 0; steps < SCREEN_HEIGHT + SCREEN_WIDTH; steps++) {
            float ray_x = camera.x + cos(theta) * steps;
            float ray_y = camera.y + sin(theta) * steps;
            if (
                ray_x ≥ SCREEN_WIDTH || ray_x < 0 ||
                ray_y ≥ SCREEN_HEIGHT || ray_y < 0 ||
                map[(int)round(ray_y)][(int)round(ray_x)] = 1
            )
                { break; }
        }

        int base_colour = 255
        if (steps ≤ 0) { steps = STEP_SIZE; }
        int shade = base colour / steps;
        int line_height = SCREEN_HEIGHT / steps + 1

        lines[ray].x = (int)round(screen_x);
        lines[ray].height = line_height;
        lines[ray].colour = shade;
    }
}
```